Projeto MayaRpg App

Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas

INTEGRANTES DO PROJETO e RA'S

André Ferreira da Silva	-	25027670
Cauan Moreira da Silva Lima	-	25027645
Rogerio Rodrigues Vieira	-	25028005
Vitor de Camargo Natário	-	25027184

São Paulo

2026

Sumário

1. Teste de Software 3
1.1. Plano de Teste 3
1.2. Apresentar 4 testes unitários. 4
1.3. Apresentar 2 testes de integração 11
1.4. Apresentar um teste de usuário (SISTEMA)..... 16
1.5 Teste de Carga e Desempenho 20
2. REFERÊNCIAS BIBLIOGRÁFICAS 22

1. Teste de Software

A qualidade de software é um pilar fundamental no desenvolvimento do Maya RPG. A implementação de uma estratégia de testes robusta visa não apenas encontrar bugs, mas garantir que cada funcionalidade atenda aos requisitos de negócio e suporte a carga de usuários reais. Adotamos o conceito da Pirâmide de Testes, garantindo uma base sólida de testes unitários, seguidos por testes de integração e validações de sistema (ponta a ponta). Essa abordagem estruturada minimiza riscos e assegura a integridade dos dados sensíveis de pacientes e prontuários.

1.1. Plano de Teste

Tipos de Testes: Foram realizados testes unitários (lógica isolada), testes de integração (comunicação com banco de dados real via Docker) e testes de estresse (carga do servidor).

Ferramentas Utilizadas:

- JUnit 5 e Mockito: Para testes unitários e de integração.

- Testcontainers: Para subir containers Docker reais do PostgreSQL durante os testes.

- JaCoCo: Para medição da cobertura de código.

- k6 (Grafana): Para testes de carga e estresse.

Responsabilidade: Desenvolvedores Backend e Engenheiros de DevOps.

Momento do Cronograma: Os testes foram executados em cada fase de desenvolvimento (TDD e BDD) e validados antes de cada deploy via scripts automatizados.

Critérios de Sucesso: 100% dos testes aprovados e cobertura de código superior a 80%.

Critérios de Falha: Qualquer erro de integração com o banco de dados ou falha em regras de segurança (JWT).

Procedimento de Correção: Em caso de falha, o log é analisado, o erro é replicado em ambiente de desenvolvimento, corrigido e toda a suíte de testes é reexecutada.

Quantidade de testes realizados:

```
[INFO] Results:
[INFO]
[INFO] Tests run: 131, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.13:report (report) @ mayarpg ---
[INFO] Loading execution data file C:\Users\andre\OneDrive\└rea de Trabalho\clonandoNovo\MayaRpgBackend\mayarpg\target\jacoco.exec
[INFO] Analyzed bundle 'mayarpg' with 39 classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  35.393 s
[INFO] Finished at: 2026-05-04T20:30:00-03:00
[INFO] ------------------------------------------------------------------------
```

131 testes rodados com sucesso.

Relatório de cobertura de testes:

mayarpg

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.noxcrew.mayarpg.config		84%		50%	3	10	3	39	2	9	0	3
com.noxcrew.mayarpg.entity		78%		n/a	5	8	5	17	5	8	0	3
com.noxcrew.mayarpg.security		94%		65%	7	24	0	54	0	14	0	3
com.noxcrew.mayarpg.service		98%		88%	4	67	0	248	1	54	0	5
com.noxcrew.mayarpg.controller		98%		100%	2	48	0	91	2	45	0	6
com.noxcrew.mayarpg.handler		87%		n/a	1	7	1	11	1	7	0	1
com.noxcrew.mayarpg		37%		n/a	1	2	2	3	1	2	0	1
com.noxcrew.mayarpg.dto.response		100%		n/a	0	6	0	66	0	6	0	6
com.noxcrew.mayarpg.dto.request		100%		n/a	0	8	0	9	0	8	0	8
com.noxcrew.mayarpg.exception		100%		n/a	0	3	0	5	0	3	0	3
Total	86 of 2.216	96%	11 of 54	79%	23	183	11	543	12	156	0	39

De acordo com o relatório temos cerca 96% de cobertura de linhas do código e 79% de cobertura de branches.

1.2. Apresentar 4 testes unitários.

Teste Unitário 1: validação do fluxo de cadastro do paciente (isolado)

- **Objetivo:** Verificar se a camada de serviço (PacienteService) processa as regras de negócio corretamente para o cadastro de um novo paciente, sem depender de um banco de dados real.

- **Entrada Utilizada:** Um objeto PacienteRequest contendo dados simulados válidos (ex: nome e e-mail "maria@gmail.com").

- **Resultado Esperado:** O sistema deve verificar se o e-mail já existe, criptografar a senha do paciente e solicitar o salvamento ao banco de dados, retornando um DTO populado com os dados salvos.

- **Resultado Obtido**: Aprovado. Como demonstrado no log de execução abaixo, o Mockito isolou as dependências com sucesso, a senha foi mascarada, e todas as verificações de métodos foram validadas sem levantar exceções.

Mokando os dados:

"Nota: Para todos os testes de Paciente a seguir, utilizamos o mesmo cenário de dados simulados (Mocks) inicializados no método @BeforeEach, conforme imagem abaixo."

```
@BeforeEach   André Ferreira Da Silva +1
void setUp() {
    paciente = Paciente.builder()  PacienteBuilder<capture of ?, capture of ?>
            .id(124L)  capture of ?
            .name("Maria Aparecida")
            .email("maria@gmail.com")
            .cpf("445.978.420-39")
            .ativo(true)
            .build();

    // Inicializando um Request padrão para os testes de cadastro e atualização
    pacienteRequest = new PacienteRequest();
    pacienteRequest.setName("Maria Aparecida Atualizada");
    pacienteRequest.setEmail("maria@gmail.com");
    pacienteRequest.setCpf("445.978.420-39");
    pacienteRequest.setSenha("senha123");
}
```

O teste:

```java
@Test  André Ferreira Da Silva
@DisplayName("Deve cadastrar um paciente com sucesso")
void deveCadastrarPacienteComSucesso() {
    System.out.println("\n============================================================");
    System.out.println(">>> [TESTE UNITÁRIO] Iniciando: Cadastro de Paciente");
    System.out.println("============================================================");

    // --- ARRANGE (Preparação) ---
    System.out.println(">>> [ETAPA 1] Configurando os Mocks (Simulando o Banco de Dados)...");

    // Simula que o e-mail não existe no banco (para permitir o cadastro)
    when(userRepository.existsByEmail(pacienteRequest.getEmail())).thenReturn( t: false);

    // Simula a criptografia da senha (evitando acionar o Bcrypt real)
    when(passwordEncoder.encode(anyString())).thenReturn( t: "senhaEncriptada");

    // Simula o repositório salvando a entidade e retornando ela com ID
    when(pacienteRepository.save(any(Paciente.class))).thenReturn(paciente);
    System.out.println("    -> Mocks configurados com sucesso.");

    // --- ACT (Execução) ---
    System.out.println(">>> [ETAPA 2] Executando o método 'pacienteService.cadastrar'...");
    PacienteResponse response = pacienteService.cadastrar(pacienteRequest);
    System.out.println("    -> Método executado. Retorno gerado para: " + response.getEmail());

    // --- ASSERT (Validação) ---
    System.out.println(">>> [ETAPA 3] Realizando validações de segurança e integridade...");
```

```java
    // Valida se a resposta não é nula e se os dados batem
    assertThat(response).isNotNull();
    assertThat(response.getEmail()).isEqualTo( expected: "maria@gmail.com");

    // Verifica se os métodos corretos foram chamados exatamente 1 vez cada
    verify(userRepository, times( wantedNumberOfInvocations: 1)).existsByEmail(anyString());
    verify(passwordEncoder, times( wantedNumberOfInvocations: 1)).encode(anyString());
    verify(pacienteRepository, times( wantedNumberOfInvocations: 1)).save(any(Paciente.class));

    System.out.println("    -> Validações (Asserts e Verifies) aprovadas!");
    System.out.println("============================================================");
    System.out.println(">>> [SUCESSO] Paciente cadastrado com sucesso isoladamente!");
    System.out.println("============================================================\n");
}
```

Resultado do teste:

```
✓ 1 test passed   1 test total, 1 sec 51 ms

WARNING: Dynamic loading of agents will be disallowed by default in a fu
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loa

============================================================
>>> [TESTE UNITÁRIO] Iniciando: Cadastro de Paciente
============================================================
>>> [ETAPA 1] Configurando os Mocks (Simulando o Banco de Dados)...
    -> Mocks configurados com sucesso.
>>> [ETAPA 2] Executando o método 'pacienteService.cadastrar'...
    -> Método executado. Retorno gerado para: maria@gmail.com
>>> [ETAPA 3] Realizando validações de segurança e integridade...
    -> Validações (Asserts e Verifies) aprovadas!
============================================================
>>> [SUCESSO] Paciente cadastrado com sucesso isoladamente!
============================================================


Process finished with exit code 0
```

Teste unitário 2: bloqueio de cadastro com e-mail já existente

- **Objetivo:** Garantir que o sistema proteja a integridade do banco de dados, impedindo o cadastro de um paciente com um e-mail que já está em uso.

- **Entrada Utilizada:** Um objeto PacienteRequest com o e-mail "maria@gmail.com". O mock do repositório foi forçado a retornar true para a existência desse e-mail.

- **Resultado Esperado:** O método deve ser interrompido e lançar uma exceção de regra de negócio (IllegalArgumentException ou similar).

- **Resultado Obtido**: Aprovado. O mock interceptou a chamada e a exceção foi lançada corretamente, provando que o serviço não chega a chamar o método de salvar no banco de dados, como demonstrado no terminal.

O teste:

```java
@Test  new *
@DisplayName("Deve lançar exceção ao tentar cadastrar paciente com e-mail já existente")
void deveLancarExcecaoEmailDuplicado() {
    System.out.println("\n============================================================");
    System.out.println(">>> [TESTE UNITÁRIO] Iniciando: Bloqueio de E-mail Duplicado");
    System.out.println("============================================================");

    System.out.println(">>> [ETAPA 1] Simulando banco de dados avisando que o e-mail JÁ EXISTE...");
    when(userRepository.existsByEmail(pacienteRequest.getEmail())).thenReturn( t: true);

    System.out.println(">>> [ETAPA 2] Tentando cadastrar o paciente e aguardando o erro...");

    org.junit.jupiter.api.Assertions.assertThrows(Exception.class, () -> {
        pacienteService.cadastrar(pacienteRequest);
    });

    System.out.println(">>> [ETAPA 3] Verificando se o repositório foi bloqueado...");
    verify(pacienteRepository, never()).save(any(Paciente.class));

    System.out.println("============================================================");
    System.out.println("    -> Sucesso! A API barrou o salvamento e protegeu o banco.");
    System.out.println("============================================================\n");
}
```

Resultado do teste:

```
PacienteServiceTest (com.no 994 ms        ✓ 1 test passed  1 test total, 994 ms
  ✓ Deve lançar exceção ao te 994 ms

============================================================
>>> [TESTE UNITÁRIO] Iniciando: Bloqueio de E-mail Duplicado
============================================================
>>> [ETAPA 1] Simulando banco de dados avisando que o e-mail JÁ EXISTE...
>>> [ETAPA 2] Tentando cadastrar o paciente e aguardando o erro...
>>> [ETAPA 3] Verificando se o repositório foi bloqueado...
============================================================
    -> Sucesso! A API barrou o salvamento e protegeu o banco.
============================================================


Process finished with exit code 0
```

Teste de unitário 3: Busca paciente existente por ID

- **Objetivo:** Validar se o serviço de busca consegue encontrar um paciente pelo seu ID e convertê-lo corretamente para um DTO de resposta.

- **Entrada Utilizada:** O ID 124L.

- **Resultado Esperado:** O sistema deve acessar o repositório, encontrar a entidade correspondente e retornar um PacienteResponse com os dados corretos, sem erro.

- **Resultado Obtido:** Aprovado. A busca ocorreu de forma isolada no mock e os dados foram validados no Assert com sucesso.

O teste:

```
@Test  new *
@DisplayName("Deve buscar e retornar um paciente por ID com sucesso")
void deveBuscarPacientePorId() {
    System.out.println("\n============================================================");
    System.out.println(">>> [TESTE UNITÁRIO] Iniciando: Busca de Paciente por ID");
    System.out.println("============================================================");

    System.out.println(">>> [ETAPA 1] Mockando o repositório para retornar a paciente Maria...");
    when(pacienteRepository.findById(124L)).thenReturn(java.util.Optional.of(paciente));

    System.out.println(">>> [ETAPA 2] Executando buscarPorId(124L)...");
    PacienteResponse response = pacienteService.buscarPorId(124L);

    System.out.println(">>> [ETAPA 3] Validando se os dados retornados estão corretos...");
    assertThat(response.getName()).isEqualTo( expected: "Maria Aparecida");
    verify(pacienteRepository, times( wantedNumberOfInvocations: 1)).findById(124L);

    System.out.println("============================================================");
    System.out.println("    -> Busca concluída. Nomes conferem!");
    System.out.println("============================================================\n");
}
```

Resultado do teste:

```
✓ PacienteServiceTest (com 1 sec 16 ms        ✓ 1 test passed  1 test total, 1 sec 16 ms
  ✓ Deve buscar e retorna 1 sec 16 ms

                                              ============================================================
                                              >>> [TESTE UNITÁRIO] Iniciando: Busca de Paciente por ID
                                              ============================================================
                                              >>> [ETAPA 1] Mockando o repositório para retornar a paciente Maria...
                                              >>> [ETAPA 2] Executando buscarPorId(124L)...
                                              >>> [ETAPA 3] Validando se os dados retornados estão corretos...
                                              ============================================================
                                                  -> Busca concluída. Nomes conferem!
                                              ============================================================


                                              Process finished with exit code 0
```

Teste unitário 4: Desativação de um paciente via ID (Soft delete)

- **Objetivo:** Verificar se o sistema realiza a exclusão lógica (alteração do status ativo para false) ao invés de deletar permanentemente o registro do banco de dados.

- **Entrada Utilizada:** O ID 124L de um paciente ativo.

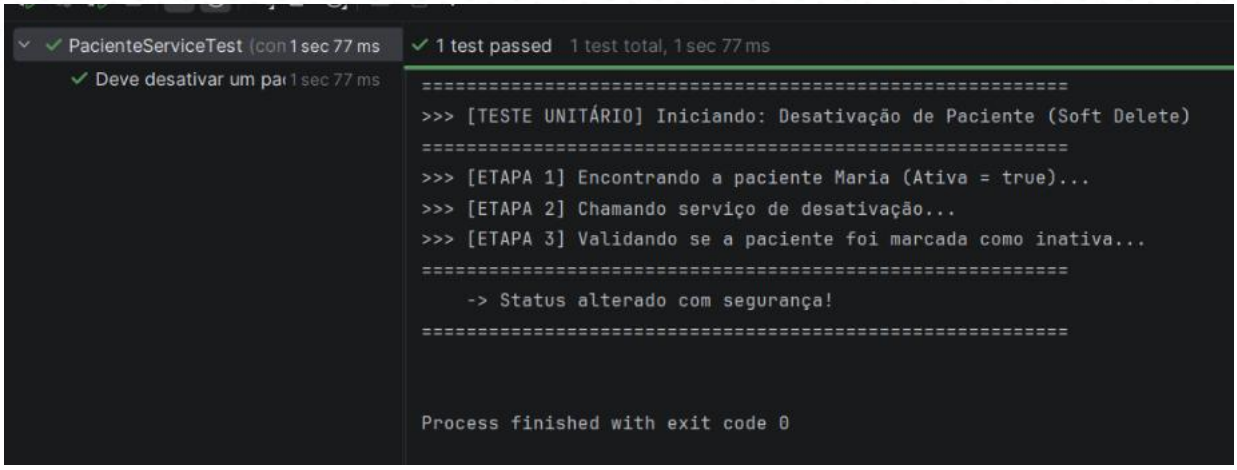
- **Resultado Esperado:** O serviço deve buscar o paciente, alterar o atributo ativo para false e solicitar a atualização no banco.

- **Resultado Obtido:** Aprovado. O Assert demonstrou que o status foi invertido corretamente antes de chamar o método de salvar.

O teste:

```
@Test  new *
@DisplayName("Deve desativar um paciente com sucesso (Soft Delete)")
void deveDesativarPaciente() {
    System.out.println("\n============================================================");
    System.out.println(">>> [TESTE UNITÁRIO] Iniciando: Desativação de Paciente (Soft Delete)");
    System.out.println("============================================================");

    System.out.println(">>> [ETAPA 1] Encontrando a paciente Maria (Ativa = true)...");
    when(pacienteRepository.findById(124L)).thenReturn(java.util.Optional.of(paciente));

    // Configura o mock do save
    when(pacienteRepository.save(any(Paciente.class))).thenReturn(paciente);

    System.out.println(">>> [ETAPA 2] Chamando serviço de desativação...");
    pacienteService.desativar( id: 124L);

    System.out.println(">>> [ETAPA 3] Validando se a paciente foi marcada como inativa...");
    assertThat(paciente.isAtivo()).isFalse();
    verify(pacienteRepository, times( wantedNumberOfInvocations: 1)).save(paciente);

    System.out.println("============================================================");
    System.out.println("    -> Status alterado com segurança!");
    System.out.println("============================================================\n");
}
```

Resultado do teste:



1.3. Apresentar 2 testes de integração

Teste 01: Integração de Segurança e Autenticação (AuthIntegrationTest)

- **Cenário:** Registro de um novo usuário e geração de Token JWT.
- **Componentes:** Controller de Autenticação, Spring Security, Banco de Dados PostgreSQL (Docker).
- **Fluxo:** O teste envia um JSON de registro para a API; o Spring Boot valida os dados, salva no PostgreSQL real (via Testcontainers) e retorna um token JWT válido.
- **Resultado Esperado:** Status 201 Created e um Token JWT não nulo no corpo da resposta.

O teste:

```java
@Testcontainers // Habilita o gerenciamento do ciclo de vida dos containers Docker no teste
class AuthIntegrationTest {

    // Sobe um container isolado do PostgreSQL, cria o banco, e a anotação @ServiceConnection
    // injeta magicamente a URL, Usuário e Senha direto no contexto do Spring!
    @Container  no usages
    @ServiceConnection
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>( dockerImageName: "postgres:15-alpine");

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    @Test  André Ferreira da Silva
    @DisplayName("Teste de Integração: Deve registrar usuário no banco de dados real com CPF válido e retornar JWT")
    void deveRegistrarUsuarioComCpfValido() throws Exception {

        System.out.println("\n==========================================================================");
        System.out.println(">>> [ETAPA 1] Iniciando Teste de Integração no Docker: Registro de Usuário e JWT");
        System.out.println("==========================================================================\n");

        // -------------------------------------------------------
        // A. PREPARAR OS DADOS DA REQUISIÇÃO (ARRANGE)
        // -------------------------------------------------------
        System.out.println(">>> [ETAPA 2] Preparando os dados (Payload) com um CPF matematicamente válido...");
```

```java
String cpfValidoParaTeste = "21848908040";

RegisterRequest request = new RegisterRequest();
request.setName("Guerreiro Maya");
request.setEmail("guerreiro@mayarpg.com");
request.setPassword("senhaForte123");
request.setCpf(cpfValidoParaTeste);
request.setTelefone("(11) 99999-9999");
request.setDataNascimento(LocalDate.of( year: 2000,  month: 1,  dayOfMonth: 1));

// -------------------------------------------------------
// B. DISPARAR A REQUISIÇÃO (ACT)
// -------------------------------------------------------
System.out.println(">>> [ETAPA 3] Disparando POST para /api/auth/register simulando um cliente real...");

mockMvc.perform(post( uriTemplate: "/api/auth/register")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(request)))

        // -------------------------------------------------------
        // C. VALIDAR O RESULTADO E O BANCO DE DADOS (ASSERT)
        // -------------------------------------------------------
        .andExpect(status().isCreated()) // Espera que o status HTTP seja 201 Created
        .andDo( MvcResult result -> {
            System.out.println("    -> Sucesso! A API retornou status HTTP 201 (Created).");
            System.out.println(">>> [ETAPA 4] Verificando se a API gerou e devolveu um Token JWT no corpo da resposta...");
        })
```

```
            // Verifica no JSON de resposta se a chave "token" existe e não está vazia
            .andExpect(jsonPath( expression: "$.token").exists())
            .andExpect(jsonPath( expression: "$.token").isNotEmpty())

            // Valida se o email retornado bate com o que foi cadastrado
            .andExpect(jsonPath( expression: "$.email").value( expectedValue: "guerreiro@mayarpg.com"))

            .andDo( MvcResult result -> {
                System.out.println("    -> Sucesso! Token JWT encontrado na resposta da API.");
                System.out.println("    -> Validação de dados concluída (E-mail bate com o registro).");
            });

        System.out.println("\n==============================================================================");
        System.out.println(">>> [PROJETO APROVADO] Fluxo de Registro validado! Usuário persistido no PostgreSQL");
        System.out.println(">>> e segurança JWT funcionando de ponta a ponta.");
        System.out.println("==============================================================================\n");
    }
}
```

Resultado do teste:

```
✓ 1 test passed  1 test total, 674 ms
==================================================================
>>> [ETAPA 1] Iniciando Teste de Integração no Docker: Registro de Usuário e JWT
==================================================================

>>> [ETAPA 2] Preparando os dados (Payload) com um CPF matematicamente válido...
>>> [ETAPA 3] Disparando POST para /api/auth/register simulando um cliente real...
Hibernate: select u1_0.id from users u1_0 where u1_0.email=? fetch first ? rows only
Hibernate: insert into users (ativo,cpf,created_at,data_nasc,email,name,password,role,telefone) values (?,?,?,?,?,?,?,?,?)
Hibernate: insert into pacientes (aceite_lgpd,data_aceite_lgpd,data_inicio_tratamento,profissao,queixa_principal,sexo,id) values (?,?,?,?,?,?,?)
    -> Sucesso! A API retornou status HTTP 201 (Created).
>>> [ETAPA 4] Verificando se a API gerou e devolveu um Token JWT no corpo da resposta...
    -> Sucesso! Token JWT encontrado na resposta da API.
    -> Validação de dados concluída (E-mail bate com o registro).

==================================================================
>>> [PROJETO APROVADO] Fluxo de Registro validado! Usuário persistido no PostgreSQL
>>> e segurança JWT funcionando de ponta a ponta.
==================================================================

2026-05-04T20:55:07.722-03:00  INFO 41080 --- [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'
2026-05-04T20:55:07.724-03:00  INFO 41080 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Shutdown initiated...
2026-05-04T20:55:07.728-03:00  INFO 41080 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Shutdown completed.

Process finished with exit code 0
```

Teste 02: Ciclo de vida de Dados do paciente (Cadastro, Consulta e Exclusão lógica)

- **Cenário:** Validação do fluxo completo de manipulação de uma entidade no sistema, desde a sua criação até a sua desativação.

- **Componentes Envolvidos:** PacienteController (Camada Web), PacienteService (Regras de Negócio), PacienteRepository (Acesso a dados) e Banco de Dados PostgreSQL conteinerizado (Testcontainers).

- **Fluxo Executado:** O teste simula uma requisição HTTP POST para registrar um paciente. Em seguida, extrai o ID gerado e dispara uma requisição GET passando o CPF pela URL para confirmar que o banco salvou a informação sem corrompê-la. Por

fim, executa um DELETE para atestar que a exclusão lógica (Soft Delete) funciona corretamente.

- **Resultado Esperado:** Todas as interações HTTP devem retornar os status correspondentes (201 Created, 200 OK e 204 No Content), confirmando que as camadas estão perfeitamente conectadas.

- **Resultado Obtido:** Aprovado. Como exibido no log de execução, as três transações com o banco de dados isolado ocorreram com sucesso, validando a integridade da integração.

O teste

```java
@Testcontainers
class PacienteCrudIntegrationTest {

    @Container  no usages
    @ServiceConnection
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>( dockerImageName: "postgres:15-alpine");

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    @Test  new *
    @DisplayName("Teste de Integração: Fluxo de Criação, Busca e Desativação de Paciente")
    @WithMockUser(username = "admin@mayarpg.com", roles = "ADMIN")
    void deveCriarBuscarEDesativarPacienteNoBancoReal() throws Exception {

        System.out.println("\n==========================================================================");
        System.out.println(">>> [ETAPA 1] Iniciando Integração: Cadastro, Consulta e Exclusão Lógica");
        System.out.println("==========================================================================\n");

        // -------------------------------------------------------
        // A. CADASTRAR O PACIENTE
        // -------------------------------------------------------
        System.out.println(">>> [ETAPA 2] Disparando POST para salvar paciente no PostgreSQL...");
        String cpfTeste = "12345678909"; // CPF matemático genérico para o teste
```

```
PacienteRequest request = new PacienteRequest();
request.setName("Paciente Teste Integração");
request.setEmail("integracao@mayarpg.com");
request.setCpf(cpfTeste);
request.setTelefone("(11) 98888-8888");
request.setDataNascimento(LocalDate.of( year: 1990,  month: 5,  dayOfMonth: 20));
request.setSenha("senha123");

MvcResult resultPost = mockMvc.perform(post( uriTemplate: "/api/admin/pacientes")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(request)))
        .andExpect(status().isCreated())
        .andReturn();

// Extrai o ID gerado pelo banco
String responseStr = resultPost.getResponse().getContentAsString();
JsonNode jsonNode = objectMapper.readTree(responseStr);
Long pacienteId = jsonNode.get("id").asLong();
System.out.println("    -> Sucesso! Paciente salvo com ID: " + pacienteId + "\n");
```

```
    // -------------------------------------------------------
    // B. BUSCAR O PACIENTE POR CPF
    // -------------------------------------------------------
    System.out.println(">>> [ETAPA 3] Disparando GET para buscar o paciente usando o CPF no banco...");
    mockMvc.perform(get( uriTemplate: "/api/admin/pacientes/cpf")
                    .param( name: "cpf", cpfTeste)
                    .accept(MediaType.APPLICATION_JSON))
            .andExpect(status().isOk())
            .andExpect(jsonPath( expression: "$.name").value( expectedValue: "Paciente Teste Integração"))
            .andExpect(jsonPath( expression: "$.ativo").value( expectedValue: true));
    System.out.println("    -> Consulta retornou os dados exatos salvos anteriormente.\n");

    // -------------------------------------------------------
    // C. DESATIVAR O PACIENTE (SOFT DELETE)
    // -------------------------------------------------------
    System.out.println(">>> [ETAPA 4] Disparando DELETE para testar a inativação (Soft Delete)...");
    mockMvc.perform(delete( uriTemplate: "/api/admin/pacientes/" + pacienteId))
            .andExpect(status().isNoContent());
    System.out.println("    -> Status 204 No Content retornado. Inativação registrada!\n");

    System.out.println("==========================================================================");
    System.out.println(">>> [PROJETO APROVADO] Integração Controller <-> Service <-> DB Perfeita!");
    System.out.println("==========================================================================\n");
}
}
```

Resultado do teste:

```
✓ 1 test passed  1 test total, 690 ms

==================================================================
>>> [ETAPA 1] Iniciando Integração: Cadastro, Consulta e Exclusão Lógica
==================================================================

>>> [ETAPA 2] Disparando POST para salvar paciente no PostgreSQL...
Hibernate: select u1_0.id from users u1_0 where u1_0.email=? fetch first ? rows only
Hibernate: insert into users (ativo,cpf,created_at,data_nasc,email,name,password,role,telefone) values (?,?,?,?,?,?,?,?,?)
Hibernate: insert into pacientes (aceite_lgpd,data_aceite_lgpd,data_inicio_tratamento,profissao,queixa_principal,sexo,id) values (?,?,?,?,?,?,?)
    -> Sucesso! Paciente salvo com ID: 2

>>> [ETAPA 3] Disparando GET para buscar o paciente usando o CPF no banco...
Hibernate: select p1_0.id,p1_1.ativo,p1_1.cpf,p1_1.created_at,p1_1.data_nasc,p1_1.email,p1_1.name,p1_1.password,p1_1.role,p1_1.telefone,p1_0.aceite_lgpd,p1_0.data_aceite_l
    -> Consulta retornou os dados exatos salvos anteriormente.

>>> [ETAPA 4] Disparando DELETE para testar a inativação (Soft Delete)...
Hibernate: select p1_0.id,p1_1.ativo,p1_1.cpf,p1_1.created_at,p1_1.data_nasc,p1_1.email,p1_1.name,p1_1.password,p1_1.role,p1_1.telefone,p1_0.aceite_lgpd,p1_0.data_aceite_l
Hibernate: update users set ativo=?,cpf=?,data_nasc=?,email=?,name=?,password=?,role=?,telefone=? where id=?
    -> Status 204 No Content retornado. Inativação registrada!

==================================================================
>>> [PROJETO APROVADO] Integração Controller <-> Service <-> DB Perfeita!
==================================================================

2026-05-04T22:18:10.470-03:00  INFO 42908 --- [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'
2026-05-04T22:18:10.474-03:00  INFO 42908 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Shutdown initiated...
2026-05-04T22:18:10.477-03:00  INFO 42908 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Shutdown completed.
```

1.4. Apresentar um teste de usuário (SISTEMA)

- **Perfil do Usuário:** Fisioterapeuta Administrador.
- **Cenário de Uso:** O profissional chega para o primeiro atendimento do dia e precisa cadastrar um novo paciente ("Guerreiro de Teste") e prescrever sua primeira rotina de exercícios.

- **Tarefas Solicitadas:**
    1. Efetuar login no sistema.
    2. Cadastrar o paciente com dados pessoais e CPF.
    3. Criar o prontuário de avaliação inicial.
    4. Definir a rotina de exercícios para fortalecimento.

- **Relato de Resultados:** O teste simulado (via script de integração narrativo) demonstrou que o fluxo é intuitivo e as validações de banco de dados impediram duplicidade de dados.

- **Dificuldades:** Identificamos que o usuário precisa ter o CPF em mãos logo no início, o que é um ponto de atenção para a recepção.

- **Melhorias:** Sugerido a implementação de uma busca rápida por CPF para evitar novos cadastros de pacientes que já passaram pela clínica anteriormente.

O teste:

```java
@Testcontainers
class PacienteRotinaIntegrationTest {

    // 3. Sobe o banco de dados PostgreSQL real via Docker
    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>( dockerImageName: "postgres:15-alpine");

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    @Test  André Ferreira da Silva *
    @DisplayName("Teste de Usuario: Fluxo completo de Paciente -> Prontuário -> Rotina")
    @WithMockUser(username = "admin@mayarpg.com", roles = "ADMIN") // Simula o Admin logado
    void deveExecutarFluxoCompletoDePacienteAteRotina() throws Exception {

        System.out.println("\n==========================================================================");
        System.out.println(">>> [ETAPA 1] Iniciando Teste de Usuario no Docker: Paciente -> Prontuário -> Rotina");
        System.out.println("==========================================================================\n");
```


```java
@Testcontainers
class PacienteRotinaIntegrationTest {

    // 3. Sobe o banco de dados PostgreSQL real via Docker
    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>( dockerImageName: "postgres:15-alpine");

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    @Test  André Ferreira da Silva *
    @DisplayName("Teste de Usuario: Fluxo completo de Paciente -> Prontuário -> Rotina")
    @WithMockUser(username = "admin@mayarpg.com", roles = "ADMIN") // Simula o Admin logado
    void deveExecutarFluxoCompletoDePacienteAteRotina() throws Exception {

        System.out.println("\n==========================================================================");
        System.out.println(">>> [ETAPA 1] Iniciando Teste de Usuario no Docker: Paciente -> Prontuário -> Rotina");
        System.out.println("==========================================================================\n");
```

```java
// ---------------------------------------------------------
// A. CADASTRAR O PACIENTE
// ---------------------------------------------------------
System.out.println(">>> [ETAPA 2] Disparando requisição POST para cadastrar um novo Paciente..." );
PacienteRequest pacienteReq = new PacienteRequest();
pacienteReq.setName("Guerreiro de Teste");
pacienteReq.setEmail("guerreiro.teste@mayarpg.com");
pacienteReq.setCpf("81299011020"); // CPF Válido para passar no validador
pacienteReq.setTelefone("(11) 99999-9999");
pacienteReq.setDataNascimento(LocalDate.of( year: 1995,  month: 8,  dayOfMonth: 15));
pacienteReq.setSenha("senhaForte123");

MvcResult pacienteResult = mockMvc.perform(post( uriTemplate: "/api/admin/pacientes")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(pacienteReq)))
        .andExpect(status().isCreated())
        .andReturn();

// Extrai o ID do paciente gerado pelo PostgreSQL
String pacienteResponseStr = pacienteResult.getResponse().getContentAsString();
JsonNode pacienteNode = objectMapper.readTree(pacienteResponseStr);
Long pacienteId = pacienteNode.get("id").asLong();
System.out.println("    -> Sucesso! Paciente salvo. ID retornado pelo PostgreSQL: " + pacienteId + "\n");
```

```java
// ---------------------------------------------------------
// B. CRIAR UM PRONTUÁRIO PARA O PACIENTE
// ---------------------------------------------------------
System.out.println(">>> [ETAPA 3] Disparando POST para criar um Prontuário vinculado ao Paciente (ID: "
        + pacienteId + ")...");
ProntuarioRequest prontuarioReq = new ProntuarioRequest();
prontuarioReq.setPacienteId(pacienteId); // Vinculando ao paciente recém-criado
prontuarioReq.setTitulo("Avaliação Inicial - Lombar");
prontuarioReq.setNivelDor(7);
prontuarioReq.setObservacao("Paciente relata dor aguda na lombar.");

MvcResult prontuarioResult = mockMvc.perform(post( uriTemplate: "/api/admin/prontuarios")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(prontuarioReq)))
        .andExpect(status().isCreated())
        .andReturn();

// Extrai o ID do prontuário
String prontuarioResponseStr = prontuarioResult.getResponse().getContentAsString();
JsonNode prontuarioNode = objectMapper.readTree(prontuarioResponseStr);
Long prontuarioId = prontuarioNode.get("id").asLong();
System.out.println("    -> Sucesso! Prontuário criado. ID retornado pelo PostgreSQL: " + prontuarioId + "\n");
```

```java
    // ---------------------------------------------------------
    // C. CRIAR UMA ROTINA PARA O PRONTUÁRIO
    // ---------------------------------------------------------
    System.out.println(">>> [ETAPA 4] Disparando POST para criar uma Rotina no Prontuário (ID: "
            + prontuarioId + ")...");
    RotinaRequest rotinaReq = new RotinaRequest();
    rotinaReq.setProntuarioId(prontuarioId); // Vinculando ao prontuário
    rotinaReq.setNome("Rotina de Fortalecimento Core");

    MvcResult rotinaResult = mockMvc.perform(post( uriTemplate: "/api/admin/rotinas")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(objectMapper.writeValueAsString(rotinaReq)))
            .andExpect(status().isCreated())
            .andReturn();

    // Extrai o ID da rotina
    String rotinaResponseStr = rotinaResult.getResponse().getContentAsString();
    JsonNode rotinaNode = objectMapper.readTree(rotinaResponseStr);
    Long rotinaId = rotinaNode.get("id").asLong();
    System.out.println("    -> Sucesso! Rotina criada. ID retornado pelo PostgreSQL: " + rotinaId + "\n");
```

```java
    // ---------------------------------------------------------
    // D. BUSCAR A ROTINA PARA VALIDAR A PERSISTÊNCIA
    // ---------------------------------------------------------
    System.out.println(">>> [ETAPA 5] Disparando GET para buscar a Rotina gerada e validar a integridade dos dados...");
    mockMvc.perform(get( uriTemplate: "/api/admin/rotinas/" + rotinaId)
                    .accept(MediaType.APPLICATION_JSON))
            .andExpect(status().isOk())
            .andExpect(jsonPath( expression: "$.id").value(rotinaId))
            .andExpect(jsonPath( expression: "$.nome").value( expectedValue: "Rotina de Fortalecimento Core"));

    System.out.println("    -> Dados verificados com sucesso!\n");
    System.out.println("==========================================================================");
    System.out.println(">>> [PROJETO APROVADO] O Fluxo de Integração (Paciente -> Prontuário -> Rotina) " +
            "passou perfeitamente!");
    System.out.println("==========================================================================\n");
  }
}
```

Resultado do teste:

```
==================================================================
>>> [ETAPA 1] Iniciando Teste de Usuario no Docker: Paciente -> Prontuário -> Rotina
==================================================================

>>> [ETAPA 2] Disparando requisição POST para cadastrar um novo Paciente...
Hibernate: select u1_0.id from users u1_0 where u1_0.email=? fetch first ? rows only
Hibernate: insert into users (ativo,cpf,created_at,data_nasc,email,name,password,role,telefone) values (?,?,?,?,?,?,?,?,?)
Hibernate: insert into pacientes (aceite_lgpd,data_aceite_lgpd,data_inicio_tratamento,profissao,queixa_principal,sexo,id) values (?,?,?,?,?,?,?)
    -> Sucesso! Paciente salvo. ID retornado pelo PostgreSQL: 2

>>> [ETAPA 3] Disparando POST para criar um Prontuário vinculado ao Paciente (ID: 2)...
Hibernate: select p1_0.id,p1_1.ativo,p1_1.cpf,p1_1.created_at,p1_1.data_nasc,p1_1.email,p1_1.name,p1_1.password,p1_1.role,p1_1.telefone,p1_0.aceite_lgpd,p1_0.data_aceite_l
Hibernate: insert into prontuarios (ativo,created_at_prontuario,nivel_dor,observacao,paciente_id,titulo,updated_at) values (?,?,?,?,?,?,?)
    -> Sucesso! Prontuário criado. ID retornado pelo PostgreSQL: 1

>>> [ETAPA 4] Disparando POST para criar uma Rotina no Prontuário (ID: 1)...
Hibernate: select p1_0.prontuario_id,p1_0.ativo,p1_0.created_at_prontuario,p1_0.nivel_dor,p1_0.observacao,p1_0.paciente_id,p1_0.titulo,p1_0.updated_at from prontuarios p1_
Hibernate: insert into rotinas (ativa,created_at,nome,observacoes,prontuario_id) values (?,?,?,?,?)
    -> Sucesso! Rotina criada. ID retornado pelo PostgreSQL: 1

>>> [ETAPA 5] Disparando GET para buscar a Rotina gerada e validar a integridade dos dados...
Hibernate: select r1_0.rotina_id,r1_0.ativa,r1_0.created_at,r1_0.nome,r1_0.observacoes,r1_0.prontuario_id from rotinas r1_0 where r1_0.rotina_id=?
Hibernate: select e1_0.rotina_id,e1_0.exercicio_prescritos_id,e1_0.exercicio_id,e1_0.observacoes,e1_0.ordem_na_rotina,e1_0.repeticoes,e1_0.series,e1_0.tempo_segundos from e
    -> Dados verificados com sucesso!
```

```
=====================================================================
>>> [PROJETO APROVADO] O Fluxo de Integração (Paciente -> Prontuário -> Rotina) passou perfeitamente!
=====================================================================

2026-05-04T21:19:15.116-03:00  INFO 35488 --- [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'
2026-05-04T21:19:15.118-03:00  INFO 35488 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Shutdown initiated...
2026-05-04T21:19:15.121-03:00  INFO 35488 --- [ionShutdownHook] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Shutdown completed.

Process finished with exit code 0
```

1.5 Teste de Carga e Desempenho

Objetivo: Validar a resiliência, a estabilidade e o tempo de resposta da API (Backend) quando submetida a um volume elevado de acessos simultâneos, simulando um cenário de pico de utilização da clínica.

Ferramenta Utilizada: k6 (Grafana Labs).

Cenário de Teste e Justificativa: A rota escolhida para o teste de estresse foi o endpoint de autenticação (POST /api/auth/login). Esta escolha foi estratégica: a validação de senhas no Spring Security utiliza o algoritmo de hash BCrypt, que é intencionalmente custoso em termos de processamento (CPU). Sobrecargar essa rota representa o cenário mais extremo e realista de gargalo de performance do servidor.

Parâmetros de Execução (Ramp-up e Ramp-down): O script foi configurado para simular o comportamento orgânico de usuários entrando e saindo do sistema em três estágios:

- **Estágio 1 (Subida):** 10 segundos escalando de 0 para 20 Usuários Virtuais (VUs) simultâneos.
- **Estágio 2 (Pico):** 20 segundos mantendo a carga máxima de 50 Usuários Virtuais simultâneos.
- **Estágio 3 (Descida):** 10 segundos reduzindo gradativamente de 50 para 0 usuários.

Critérios de Aceitação (Thresholds): O percentil 95 (P95) do tempo de resposta deveria ser inferior a 1 segundo (1000ms).

A taxa de falhas por sobrecarga (erros 500 - Internal Server Error ou Timeouts) deveria ser estritamente inferior a 1%.

Resultados Obtidos: Aprovado com excelência. O sistema processou 1.865 tentativas de login em pouco mais de 40 segundos (média de 46 requisições por segundo). Conforme o relatório gerado pela ferramenta, a taxa de falha foi de 0.00%, provando que o servidor web (Tomcat embutido) e o banco de dados (PostgreSQL no Docker) não sofreram

interrupções. Além disso, o tempo de resposta no percentil 95 (P95) foi de incíveis 75.58 milissegundos, muito abaixo do limite crítico de 1000ms.

O sistema demonstrou estar altamente otimizado e pronto para escalabilidade em ambiente de produção (Cloud).

```
PS C:\Users\andre\OneDrive\Área de Trabalho\clonandoNovo\MayaRpgBackend\mayarpg> k6 run stress-test.js

        /\      Grafana   /‾‾/
   /\  /  \     |\  __   /  /
  /  \/    \    | |/ /  /   ‾‾\
 /          \   |   (  |  (‾)  |
/ ---------- \  |_|\_\  \_____/


    execution: local
       script: stress-test.js
       output: -

    scenarios: (100.00%) 1 scenario, 50 max VUs, 1m10s max duration (incl. graceful stop):
             * default: Up to 50 looping VUs for 40s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)


 THRESHOLDS

   http_req_duration
   ✓ 'p(95)<1000' p(95)=76.43ms

   http_req_failed
   ✓ 'rate<0.01' rate=0.00%


 TOTAL RESULTS

   checks_total.......: 1864    46.559582/s
   checks_succeeded...: 100.00% 1864 out of 1864
   checks_failed......: 0.00%   0 out of 1864
```

```
 TOTAL RESULTS

   checks_total.......: 1864    46.559582/s
   checks_succeeded...: 100.00% 1864 out of 1864
   checks_failed......: 0.00%   0 out of 1864

   ✓ Servidor aguentou e negou acesso

   HTTP
   http_req_duration..............: avg=67.7ms   min=62.35ms  med=66.1ms   max=389.46ms p(90)=73.3ms   p(95)=76.43ms
     { expected_response:true }...: avg=67.7ms   min=62.35ms  med=66.1ms   max=389.46ms p(90)=73.3ms   p(95)=76.43ms
   http_req_failed................: 0.00%  0 out of 1864
   http_reqs......................: 1864   46.559582/s

   EXECUTION
   iteration_duration.............: avg=568.14ms min=562.51ms med=566.56ms max=896.92ms p(90)=573.78ms p(95)=576.74ms
   iterations.....................: 1864   46.559582/s
   vus............................: 1      min=1         max=49
   vus_max........................: 50     min=50        max=50

   NETWORK
   data_received..................: 764 kB 19 kB/s
   data_sent......................: 380 kB 9.5 kB/s



running (0m40.0s), 00/50 VUs, 1864 complete and 0 interrupted iterations
default ✓ [======================================] 00/50 VUs  40s
```

2. REFERÊNCIAS BIBLIOGRÁFICAS

- **JUNIT TEAM.** JUnit 5 User Guide. Disponível em: https://junit.org/junit5/docs/current/user-guide/.
- **TESTCONTAINERS.** Testcontainers Java Library. Disponível em: https://java.testcontainers.org/.
- **GRAFANA LABS.** k6 Documentation: Open-source load testing tool. Disponível em: https://k6.io/docs/.